

Prova III - Teoria

Entrega 15 dez em 10:40**Pontos** 10**Perguntas** 32**Disponível** 15 dez em 8:40 - 15 dez em 10:40 aproximadamente 2 horas**Limite de tempo** 120 Minutos

Instruções

Esta prova TEÓRICA vale 10 pontos. Ela tem 30 questões de verdadeiro ou falso e duas questões abertas de código. As questões de verdadeiro ou falso valem 0,2 e as abertas, 2.5. O somatório dessas notas é igual a 10. Fizemos isso porque, novamente, NÃO será permitido retornar nas questões: nem fechadas, nem abertas. A submissão das questões abertas será efetuada em campo de arquivo texto.

A prova teórica da manhã acontecerá de 8:50 às 10:30 e a da tarde, de 13:30 às 15:10. A prova será aberta 10 minutos mais cedo e fechará 10 minutos mais tarde.

Histórico de tentativas

	Tentativa	Tempo	Pontuação
MAIS RECENTE	Tentativa 1	96 minutos	5,4 de 10 *

* Algumas perguntas ainda não avaliadas

⚠ As respostas corretas estarão disponíveis de 15 dez em 15:20 a 17 dez em 15:20.

Pontuação deste teste: **5,4** de 10 *

Enviado 15 dez em 10:16

Esta tentativa levou 96 minutos.

Pergunta 1**0,2 / 0,2 pts**

Considere uma tabela *hash* com *rehash* contendo duas posições e usando as funções de transformação apresentadas abaixo, podemos afirmar que após a inserção dos elementos 1, 4, 5, 2, e 7, a tabela resultante será [4 1].

```
int hash(int key){  
    return key%2;  
}  
  
int rehash(int key){  
    return ++key%2;  
}
```

☒ Verdadeiro

☐ Falso

A afirmação é verdadeira, pois o 5, 2 e 7 não serão inseridos.

Incorreta**Pergunta 2****0 / 0,2 pts**

Os métodos *hashing* envolvem um processo de transformação de uma chave em um endereço. Sobre esses métodos, podemos afirmar que o tempo gasto com pesquisas depende do tamanho da tabela e da quantidade de elementos inseridos. Além disso, a função *hash* de transformação deve envolver uma operação simples sobre a chave (TCE-RS'14, adaptada).

☒ Falso

☐ Verdadeiro

A afirmação é verdadeira. Primeiro, a dependência do tamanho da tabela e da quantidade de elementos acontece porque tais fatores influenciam nas colisões e o quão maior o número de colisões, maior o custo para encontrar elementos. Segundo, é importante que a operação de transformação seja simples, evitando o *overhead* de processamento.

Pergunta 3

0,2 / 0,2 pts

Uma estrutura de dados eficiente é a tabela *hash*. Um desafio no projeto dessa estrutura é seu dimensionamento. Em uma tabela adequadamente dimensionada, com n chaves, o número médio de acessos para localização de uma chave é $\Theta(1)$ comparações (TJ-PI'15, adaptada).

☐ Falso

☒ Verdadeiro

Uma tabela *hash* é bem dimensionada quando seu custo é constante, $\Theta(1)$ comparações.

Pergunta 4

0,2 / 0,2 pts

Considere uma tabela *hash* contendo um *array* de árvores alvinegras. O melhor caso acontece com custo $\Theta(1)$ e o pior, com o custo $\Theta(n)$.

☐ Verdadeiro

☒ Falso

A afirmação é falsa. O melhor caso acontece quando o elemento desejado está na raiz da árvore representando sua posição. O pior caso acontece quando todos os elementos inseridos possuem o mesmo valor na função de transformação e, nesse caso, o elemento desejado encontra-se em uma das folhas da árvore ou o mesmo não está contido na árvore.

Pergunta 5**0,2 / 0,2 pts**

Considere um arquivo sequencial, com 10.000 registros, cujas chaves identificadoras são números inteiros de até 8 dígitos. Para criar um índice tipo *hashing* para esse arquivo, contendo endereços de 0 até 11.999, uma definição adequada para uma função de *hashing* $f(x)$, onde x é uma chave e $(a \bmod b)$ é o resto da divisão de a por b , seria $f = x \bmod 12000$ (DPE-RJ'14, adaptado).

☐ Falso☒ Verdadeiro

A afirmação é verdadeira. Uma função de transformação tradicional consiste no resto da divisão inteira do elemento pelo tamanho da tabela.

Pergunta 6**0,2 / 0,2 pts**

Na árvore 2.3.4, após uma inserção utilizando fragmentação na descida, podemos garantir a inexistência de nós do tipo 4 no caminho entre a raiz e a folha em que aconteceu a inserção.

☒ Falso

☐ Verdadeiro

A afirmação é falsa. Na inserção com fragmentação na descida, quando chegamos em um nó do tipo 4, esse é fragmentado. Nessa fragmentação, se o pai era do tipo 3, ele ficará sendo do tipo 4. Da mesma forma, se inserirmos em uma folha do tipo 3, essa ficará sendo do tipo 4.

Pergunta 7

0,2 / 0,2 pts

A inserção dos números 2, 12, 5, 8, 1, 11, 9, 7, 4, 13, 10, 6 e 3 em uma 2.3.4 usando as técnicas de fragmentação por ascensão e na descida gera a mesma árvore resultante.

☐ Falso

☒ Verdadeiro

A afirmação é verdadeira. A inserção usando as duas técnicas é igual até a inserção do 10. Elas ficam diferentes após a inserção do 6 e voltam a ficar iguais com a inserção do 3.

Pergunta 8**0,2 / 0,2 pts**

Ao fragmentarmos um nó do tipo 4, a altura da árvore aumenta em um nível.

☒ Falso☐ Verdadeiro

A afirmação é falsa. Nas fragmentações, o elemento do meio "sobe" para seu pai sem aumentar o nível da árvore. A única exceção acontece quando temos a fragmentação da raiz, pois, como não existe um pai, esse é criado.

Pergunta 9**0,2 / 0,2 pts**

Dadas duas árvores alvinegras A e B obtidas a partir de uma árvore 2.3.4, A e B possuem os mesmos números de nós e arestas.

☐ Falso☒ Verdadeiro

A afirmação é verdadeira. O número de nós na alvinegra é igual ao número de elementos existentes na 2.3.4. O número de arestas da alvinegra é igual à soma entre o número de arestas da 2.3.4 e o de relacionamento entre gêmeos da 2.3.4.

Pergunta 10

0,2 / 0,2 pts

A árvore 2.3.4 é uma estrutura de pesquisa cujos nós são de três tipos (2-nó, 3-nó ou 4-nó) e as folhas estão situadas no mesmo nível.

☐ Falso

☒ Verdadeiro

A afirmação é verdadeira dado que os três tipos de nós foram citados e nesta árvore sempre inserimos novos valores nas folhas.

Pergunta 11

0,2 / 0,2 pts

Uma das estruturas de dados utilizadas na modelagem de sistemas de software denomina-se árvore rubro-negra. Nesse caso, um nó será vermelho quando, na árvore 2-3-4 correspondente, o valor desse nó for gêmeo do nó pai. Caso contrário, o nó é preto. Em uma árvore rubro-negra temos que a quantidade de nós vermelhos é sempre par.

☐ Verdadeiro

☒ Falso

A afirmação é falsa conforme os conceitos de árvores alvinegras apresentados na sala de aula.

Incorreta

Pergunta 12

0 / 0,2 pts

Em uma árvore alvinegra, após uma rotação, o NÓ central será branco e, seus, filhos, pretos.

☒ Falso

☐ Verdadeiro

A afirmação é verdadeira conforme os conceitos de árvores alvinegras apresentados na sala de aula.

Pergunta 13

0,2 / 0,2 pts

Uma das estruturas de dados utilizadas na modelagem de sistemas de software denomina-se árvore rubro-negra. Nesse caso, um nó será vermelho quando, na árvore 2-3-4 correspondente, o valor desse nó for gêmeo do nó pai. Caso contrário, o nó é preto. Em uma árvore rubro-negra temos que a quantidade de nós vermelhos é sempre igual à de pretos.

☒ Falso

☐ Verdadeiro

A afirmação é falsa conforme os conceitos de árvores alvinegras apresentados na sala de aula.

Pergunta 14

0,2 / 0,2 pts

Em uma árvore alvinegra, quando um nó tem 2 filhos pretos, fazemos a inversão das cores entre os filhos e o pai.

☐ Falso

☒ Verdadeiro

A afirmação é verdadeira conforme os conceitos de árvores alvinegras apresentados na sala de aula.

Pergunta 15

0,2 / 0,2 pts

Uma árvore rubro-negra possui 18 valores inteiros distintos armazenados em seus 18 nós. A função recursiva boolean busca (int val) visita os nós desse tipo de árvore à procura de um determinado valor (val). O algoritmo utilizado tira partido das características de uma árvore rubro-negra, com o objetivo de ser o mais eficiente possível. Podemos afirmar que o número máximo de chamadas da função que será necessário para informar se um determinado valor está, ou não, armazenado na árvore é igual a seis (BNDES'13, adaptada).

☐ Falso

☒ Verdadeiro

A afirmação é verdadeira e pode ser confirmada executando o código disponibilizado pelo professor.

Pergunta 16

0,2 / 0,2 pts

A AVL é uma árvore de busca autobalanceada. Isso significa que as alturas das duas sub-árvores a partir de cada nó diferem no máximo em uma unidade (SEFAZ-PI'15, adaptado).

☐ Falso

☒ Verdadeiro

A afirmação é verdadeira conforme o conceito de árvore AVL.

Pergunta 17

0,2 / 0,2 pts

Todos os n nomes de uma lista de assinantes de uma companhia telefônica foram inseridos, em ordem alfabética, em três estruturas de dados: uma árvore binária de busca, uma árvore AVL e uma árvore bicolor.

As alturas resultantes das três árvores são $\Theta(n)$, $\Theta(\lg(n))$ e $\Theta(\lg(n))$, respectivamente (PETROBRAS'12, adaptada)

☒ Verdadeiro

☐ Falso

A afirmação é verdadeira. A AB não é balanceada e as demais são.

Pergunta 18

0,2 / 0,2 pts

A AVL é uma árvore de busca autobalanceada. Isso significa que cada nó da árvore possui até três descendentes (SEFAZ-PI'15, adaptado).

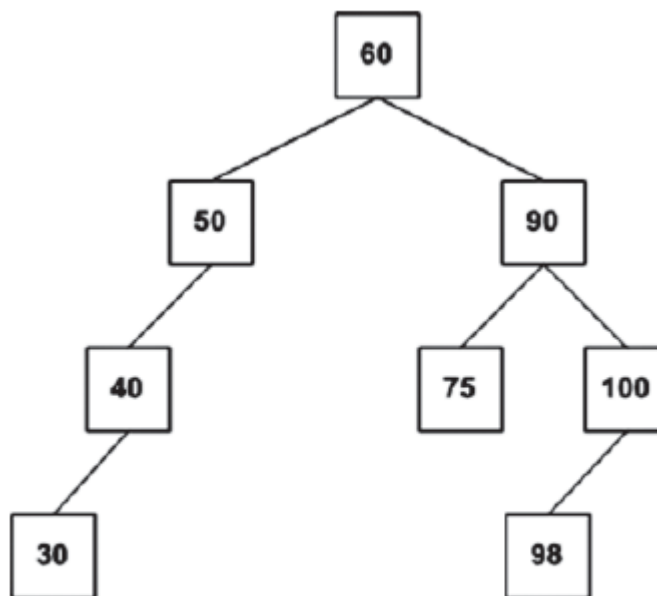
☐ Verdadeiro

☒ Falso

A afirmação é falsa conforme o conceito de árvore AVL.

Pergunta 19**0,2 / 0,2 pts**

A estrutura de dados AVL é uma árvore binária de busca balanceada criada pelos soviéticos Adelson, Velsky e Landis em 1962. Podemos afirmar que a figura abaixo representa uma árvore AVL (PETROBRAS'12, adaptada).

☐ Verdadeiro☒ Falso

A afirmação é falsa conforme o conceito de árvore AVL.

Pergunta 20

0,2 / 0,2 pts

Avaliando os códigos abaixo nas linguagens C e C++, é correto afirmar que os dois imprimem os mesmos valores para as variáveis "x" e "y" no final de suas execuções. Isso acontece mesmo com a passagem de parâmetros do primeiro código sendo por valor e a do segundo, sendo por referência.

Código em C:

```
void troca(double *x, double *y){  
    double aux = *x;  
    *x = *y;  
    *y = aux;  
}
```

```
int main(){  
    double x = 7.0;  
    double y = 5.0;  
    troca(&x, &y);  
    printf("X: %lf Y: %lf", x, y);  
}
```

Código em C++:

```
void troca(double& x, double& y){  
    double aux = x;
```

```
x = y;  
y = x;  
}  
  
int main(){  
    double x = 7.0;  
    double y = 5.0;  
    troca(x, y);  
    cout << "X: " << x;  
    cout << "Y: " << y;  
}
```

☒ Verdadeiro

☐ Falso

A afirmação é verdadeira conforme discutido em sala de aula.

Pergunta 21

0,2 / 0,2 pts

Podemos afirmar que o código abaixo em C++ imprime "1 1" como saída


```
void f (int val, int& ref) {  
    val ++; ref++;  
}
```

```
void main () {  
    int i = 1, j = 1;  
    f(i, j);  
    printf("%i -- %i", i, j);  
}
```

☒ Falso

☐ Verdadeiro

A afirmação é falsa. A variável ref foi passada por referência, assim, seu valor final é dois.

Pergunta 22

0,2 / 0,2 pts

Considere a assinatura das duas funções abaixo na linguagem C++:

```
void troca(double &x, double &y)
```

```
void troca(double* x, double* y)
```

Podemos afirmar que a primeira função utiliza passagem de parâmetros por referência e a segunda, por valor.

☐ Falso

☒ Verdadeiro

A afirmação é verdadeira considerando os conceitos sobre passagem de parâmetros na linguagem C++ apresentados na sala de aula.

Pergunta 23

0,2 / 0,2 pts

As rotações são utilizadas no balanceamento das árvores de tal forma que uma árvore desbalanceada para a esquerda será rotacionada para à direita. Uma desbalanceada para a direita será para à esquerda. Nós temos quatro tipos de rotações: simples para esquerda, dupla esquerda-direita, simples para à direita e a dupla direita-esquerda. As duas primeiras são efetuadas em árvores desbalanceadas para à esquerda e as outras duas, desbalanceadas para à direita.

☒ Falso

☐ Verdadeiro

A afirmação é falsa. As rotações simples para esquerda e a dupla direita-esquerda são efetuadas em árvores desbalanceadas para à esquerda e as outras duas, desbalanceadas para à direita.

Pergunta 24

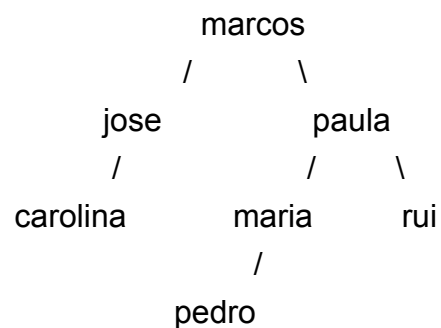
0,2 / 0,2 pts

Considere que a Manausprev armazena os nomes dos beneficiários de aposentadorias em uma Árvore Binária de Busca (ABB). Ao se armazenar, nesta ordem, os nomes Marcos, José, Carolina, Paula, Rui, Pedro e Maria, a ABB resultante é uma árvore completa (MANAUSPREV'15, adaptada).

☐ Verdadeiro

☒ Falso

A afirmação é falsa e a árvore resultante é mostrada abaixo.



Pergunta 25

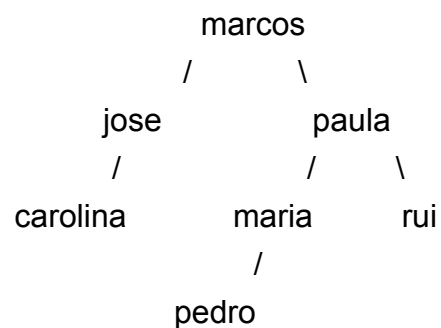
0,2 / 0,2 pts

Considere que a Manausprev armazena os nomes dos beneficiários de aposentadorias em uma Árvore Binária de Busca (ABB). Ao se armazenar, nesta ordem, os nomes Marcos, José, Carolina, Paula, Rui, Pedro e Maria, a ABB resultante tem o nó contendo José possui dois filhos (MANAUSPREV'15, adaptada).

☐ Verdadeiro

☒ Falso

A afirmação é falsa e a árvore resultante é mostrada abaixo.



Pergunta 26

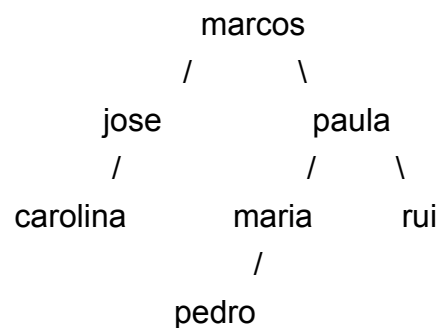
0,2 / 0,2 pts

Considere que a Manausprev armazena os nomes dos beneficiários de aposentadorias em uma Árvore Binária de Busca (ABB). Ao se armazenar, nesta ordem, os nomes Marcos, José, Carolina, Paula, Rui, Pedro e Maria, a ABB resultante tem altura 3, que corresponde à altura mínima para armazenar os 7 nomes (MANAUSPREV'15, adaptada).

☐ Verdadeiro

☒ Falso

A afirmação é falsa e a árvore resultante é mostrada abaixo.



Pergunta 27

0,2 / 0,2 pts

Nas linguagem Java e C++, ao instanciarmos um objeto usando new, fazemos com que tal objeto seja acessível a partir de um ponteiro/referência. Entretanto, diferente do Java, a linguagem C++ também permite a criação literal de objetos.

☒ Verdadeiro

☐ Falso

Nas linguagem Java e C++, o comando new cria um objeto e retorna seu endereço de memória que deve ser armazenado em um ponteiro/referência. A linguagem Java não permite a criação literal de um objeto o que é possível em C++.

Incorreta**Pergunta 28****0 / 0,2 pts**

O comando Java "No no = new No()" cria um objeto do tipo No e armazena seu endereço na variável no. A mesma coisa acontece em C++ no código "No* no = new No()".

☒ Falso☐ Verdadeiro

Ambos os comandos criam um ponteiro (ou referência) chamada "no" que armazena um endereço de memória. Além disso, o "new No()" cria um objeto de tipo No e retorna seu endereço de memória.

Pergunta 29**0,2 / 0,2 pts**

O código C++ apresentado abaixo está errado, pois acessamos o atributo de Box2 usando o símbolo “.”.

```
class Box {
    public:
        double length; // Length of a box
        double breadth; // Breadth of a box
        double height; // Height of a box
};

int main() {
    Box Box1;    // Declare Box1 of type Box
    Box Box2;    // Declare Box2 of type Box
    double volume = 0.0; // Store the volume of a box here

    // box 1 specification
    Box1->height = 5.0;
    Box1->length = 6.0;
    Box1->breadth = 7.0;

    // box 2 specification
    Box2.height = 10.0;
    Box2.length = 12.0;
    Box2.breadth = 13.0;

    // volume of box 1
    volume = Box1->height * Box1->length * Box1->breadth;
    cout << "Volume of Box1 : " << volume << endl;

    // volume of box 2
    volume = Box2.height * Box2.length * Box2.breadth;
```



```
cout << "Volume of Box2 : " << volume << endl;  
return 0;  
}
```

☐ Verdadeiro

☒ Falso

O código C++ apresentado, realmente, está errado. Contudo, o erro acontece porque acessamos os atributos de Box1 usando o símbolo "->". Box1 é um objeto e está sendo usado como se fosse um ponteiro.

Pergunta 30

0,2 / 0,2 pts

Nas linguagens C/C++/Java, acessamos um atributo de um registro/objeto apontado por um ponteiro/referência fazendo ponteiro.atributo.

☐ Verdadeiro

☒ Falso

A afirmação está correta para a linguagem Java, contudo, nas linguagens C/C++, devemos usar o símbolo "->".

Pergunta 31

Não avaliado ainda / 2 pts

Seja a árvore Trie apresentada na sala de aula implemente um método para PESQUISAR palavras na árvore. Considere que a classe Nó da árvore Trie contém uma LISTA FLEXÍVEL para armazenar o endereço dos nós filhos. Apresente a ordem de complexidade do seu método. Sua resposta deve conter somente o método solicitado e sua complexidade.

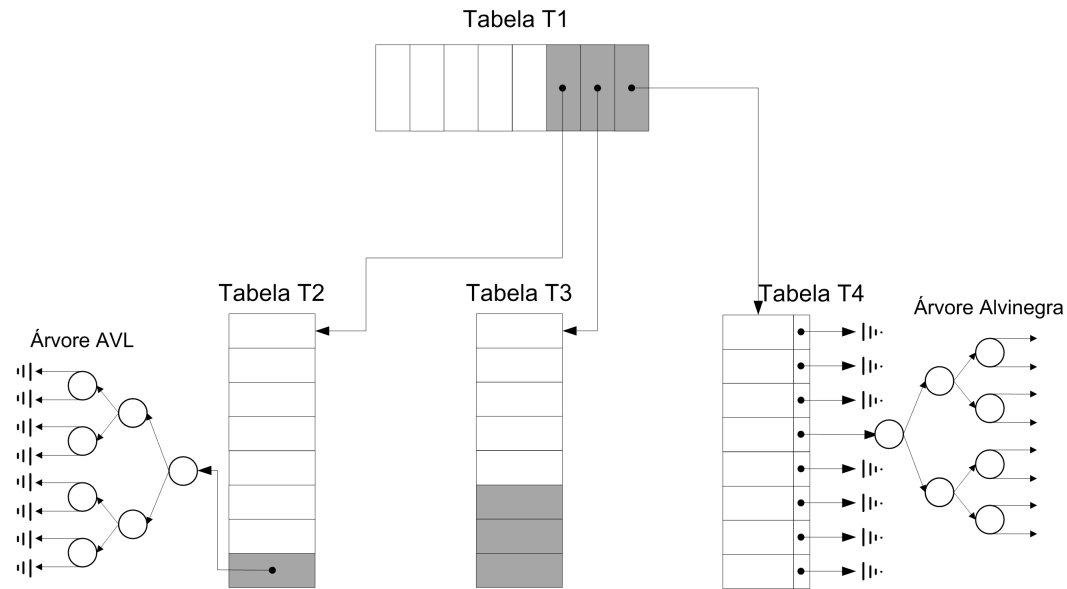
↓ [Q31.java \(https://pucminas.instructure.com/files/5195974/download\)](https://pucminas.instructure.com/files/5195974/download)

Pergunta 32

Não avaliado ainda / 2 pts

Crie a estrutura de dados Doidona (mostrada na figura abaixo) para armazenar números inteiros. Nesta questão, você deve implementar (em Java) a classe Doidona com seus atributos, um construtor e o método public void mostrar(). A estrutura doidona tem quatro tabelas. T1 é uma hash com área de reserva. T2 é uma hash com duas funções de rehash e uma área de reserva que corresponde a uma árvore AVL. T3 é uma hash com área de reserva de tamanho três. T4 é uma hash indireta com árvores alvinegras. Suponha que todos os métodos de hash e rehash estão implementados (int hashT1(int elemento), int hashT2(int

elemento), int hashT3(int elemento) e int hashT4(int elemento), int rehash1T2(int elemento), int rehash2T2(int elemento), respectivamente.



↓ [Doidona.java \(https://pucminas.instructure.com/files/5196177/download\)](https://pucminas.instructure.com/files/5196177/download)

Pontuação do teste: **5,4** de 10